

Practical 1: Preparing Software Requirements Specifications

Objectives:

- Identify ambiguities, inconsistencies and incompleteness from a requirements specification
- Identify and state functional requirements
- Identify and state non-functional requirements

Requirements:

Sommerville defines "requirement" as a specification of what should be implemented. Requirements specify how the target system should behave. It specifies what to do, but not how to do. Requirements engineering refers to the process of understanding what a customer expects from the system to be developed, and to document them in a standard and easily readable and understandable format. This documentation will serve as reference for the subsequent design, implementation and verification of the system.

It is necessary and important that before we start planning, design and implementation of the software system for our client, we are clear about its requirements. If we don't have a clear vision of what is to be developed and what all features are expected, there would be serious problems and customer dissatisfaction as well.

Characteristics of Requirements:

Requirements gathered for any new system to be developed should exhibit the following three properties:

- **Unambiguity:** There should not be any ambiguity what a system to be developed should do. For example, consider you are developing a web application for your client. The client requires that enough number of people should be able to access the application simultaneously. What's the "enough number of people"? That could mean 10 to you, but, perhaps, 100 to the client. There's an ambiguity.
- **Consistency:** To illustrate this, consider the automation of a nuclear plant. Suppose one of the clients say that if the radiation level inside the plant exceeds R1, all reactors should be shut down. However, another person from the client side suggests that the threshold radiation level should be R2. Thus, there is an inconsistency between the two end users regarding what they consider as threshold level of radiation.
- **Completeness:** A particular requirement for a system should specify what the system should do and also what it should not. For example, consider a software to be developed for ATM. If a customer enters an amount greater than the maximum permissible withdrawal amount, the ATM should display an error message, and it should not dispense any cash.

Categorization of Requirements:

Based on the target audience or subject matter, requirements can be classified into different types, as stated below:

- **User requirements:** They are written in natural language so that both customers can verify their requirements have been correctly identified
- **System requirements:** They are written involving technical terms and/or specifications, and are meant for the development or testing teams

Requirements can be classified into two groups based on what they describe:

- **Functional requirements (FRs):** These describe the functionality of a system -- how a system should react to a particular set of inputs and what should be the corresponding output.
- **Non-functional requirements (NFRs):** They are not directly related what functionalities are expected from the system. However, NFRs could typically define how the system should behave under certain situations. For example, a NFR could say that the system should work with 128MB RAM. Under such condition, a NFR could be more critical than a FR.

Non-functional requirements could be further classified into different types like:

- **Product requirements:** For example, a specification that the web application should use only plain HTML, and no frames
- **Performance requirements:** For example, the system should remain available 24x7
- **Organizational requirements:** The development process should comply to SEI CMM level 4

Functional Requirements:

Identifying Functional Requirements

Given a problem statement, the functional requirements could be identified by focusing on the following points:

- Identify the high level functional requirements simply from the conceptual understanding of the problem. For example, a Library Management System, apart from anything else, should be able to issue and return books.
- Identify the cases where an end user gets some meaningful work done by using the system. For example, in a digital library a user might use the "Search Book" functionality to obtain information about the books of his interest.
- If we consider the system as a black box, there would be some inputs to it, and some output in return. This black box defines the functionalities of the system. For example, to search for a book, user gives title of the book as input and get the book details and location as the output.
- Any high level requirement identified could have different sub-requirements. For example, "Issue Book" module could behave differently for different class of users, or for a particular user who has issued the book thrice consecutively.

Preparing Software Requirements Specifications(SRS):

Once all possible FRs and non-FRs have been identified, which are complete, consistent, and non-ambiguous, the Software Requirements Specification (SRS) is to be prepared. IEEE provides a template, which could be used for this purpose. The SRS is prepared by the service provider, and verified by its client. This document serves as a legal agreement between the client and the service provider. Once the concerned system has been developed and deployed, and a proposed feature was not found to be present in the system, the client can point this out from the SRS. Also, if after delivery, the client says a new feature is required, which was not mentioned in the SRS, the service provider can again point to the SRS. The scope of the current experiment, however, doesn't cover writing a SRS.

Case Study: A Cloud Gaming Console Platform for GameSphere Technologies

Background:

GameSphere Technologies is a startup focused on providing cloud-based gaming services to users who do not own high-end gaming hardware. With the increasing popularity of online gaming and advancements in cloud computing, the company plans to launch a Cloud Gaming Console Platform that enables users to play games directly through a web browser without downloading or installing them.

The platform hosts games on powerful cloud servers, processes all game execution remotely, and streams the gameplay to users over the internet. Users only need a stable internet connection and a compatible device such as a laptop, desktop, tablet, or smartphone.

The proposed system will manage user accounts, game libraries, cloud gaming sessions, subscriptions, cloud save data, and administrative operations. The platform aims to provide a seamless gaming experience while efficiently managing cloud resources.

System Overview:

The Cloud Gaming Console allows users to:

- Register and create an account.
- Log in securely.
- Browse and search the game catalog.
- Purchase or subscribe to gaming plans.
- Launch games hosted on cloud servers.
- Save and resume game progress from any device.
- View achievements and gaming history.
- Manage account settings.

Administrators can:

- Add or remove games.
- Manage users and subscriptions.
- Monitor active gaming sessions.

- View platform statistics.
- Allocate and manage cloud gaming servers.

Visitors who are not registered can browse available games but cannot launch or save games until they create an account.

Identification of Functional Requirements:

1. User Registration

A new user can register by providing:

- Full Name
- Email Address
- Username
- Password

After successful registration, the system creates a unique User ID and stores the account securely.

2. User Login

Registered users can log in using their email or username and password.

If invalid credentials are entered three consecutive times:

- The account is temporarily locked.
- A password reset link is sent to the registered email.

Only authenticated users can access gaming features.

3. Browse and Search Games

Both visitors and registered users can search games using:

- Game Title
- Genre
- Developer
- Platform
- Release Year

4. The system displays:

- Game description
 - Supported devices
 - Rating
 - Subscription requirement
-

5. Launch Game

A logged-in user can launch a game only if:

- The user has an active subscription.
- A cloud gaming server is available.
- The selected game is available.

The system then:

- Allocates a cloud server.
 - Creates a gaming session.
 - Starts streaming the game.
 - Displays connection quality information.
-

6. Cloud Save

Game progress is automatically saved to cloud storage.

When users launch the same game from another device, they can continue from their last saved checkpoint.

7. Subscription Management

Users can:

- View available subscription plans.
- Upgrade or downgrade plans.
- Renew expired subscriptions.
- View billing history.

Only active subscriptions allow access to premium games.

8. Gaming History

The system records:

- Recently played games
- Total playtime
- Last login
- Session history
- Achievements earned

9. Administrator Management

The administrator can:

- Add new games.
- Remove unavailable games.
- Update game information.
- Manage user accounts.
- Suspend or reactivate accounts.
- View server utilization.
- Generate usage reports.

TABLE 01: IDENTIFIER AND PRIORITY FOR SOFTWARE REQUIREMENTS

#	Requirement	Priority
R1	User Registration	High
R2	User Login	High
R3	Browse/Search Games	High
R4	Launch Cloud Game	High
R5	Cloud Save Management	High
R6	Cloud Save Management	Low
R7	Gaming History	Medium
R8	Admin Dashboard	High

10. Identification of Non-Functional Requirements:

→ Performance Requirements

- The platform shall be available 24×7.
 - At least 200 users should be able to use the system simultaneously.
 - A cloud gaming session should begin within 10 seconds after clicking Play.
 - Search results should appear within 2 seconds.
-

→ Security Requirements

- Passwords must be stored using secure hashing algorithms.
 - All communication between client and server must use HTTPS.
 - Only authenticated users can launch games.
 - User save data must be protected from unauthorized access.
 - Sessions should automatically expire after prolonged inactivity.
-

→ Reliability Requirements

- Cloud save data should not be lost in case of unexpected disconnections.
 - Failed gaming sessions should be recoverable whenever possible.
-

→ Usability Requirements

- The interface should be responsive for desktops, tablets, and smartphones.
 - Navigation should be simple and intuitive.
 - Games should be searchable using multiple filters.
-

→ Design Constraints

- The application shall be developed as a web-based platform using modern HTML5 technologies.
- The frontend should support current versions of major web browsers.
- The backend shall expose REST APIs for communication.

- The system should use a relational database for user, subscription, and gaming records.
-

→ **Software Quality Attributes**

- Scalability to support increasing numbers of users.
- Maintainable modular architecture.
- High availability of cloud gaming services.
- Secure authentication and authorization.
- Efficient resource allocation for cloud gaming sessions.